Approach 1 — STT output vs. gold transcript

Transcribe an audio file with Whisper, compare the result against a manual **gold transcript** (`.pdf`/`.txt`), and report what the STT got wrong: **missing**, **incorrect**, **conflicting**, or **hallucinated** information.

The comparison has two layers:

1. A quick **deterministic** pass (word/character similarity, coverage, entity checks) that resolves the easy cases cheaply.
2. An **LLM judge** (Gemini) that reads the gold and candidate segments and classifies the hard ones into one of the four categories above.

No API key? The pipeline falls back to its built-in heuristics and still runs offline (just less accurate).

How it works

```
audio ─▶ Whisper ─▶ candidate transcript
gold transcript ──────▶  segment + match the two texts
                           │
                           ├─ deterministic signals (cheap checks)
                           └─ LLM judge: anything not "match"
                               → missing / incorrect / conflict /
hallucinated
                           │
                           review pass (remove false alarms)
                           ▼
                     overall score → Match or Mismatch
```

Setup

Use the repo's virtual environment and install dependencies:

```
source ../.venv/bin/activate
pip install -r requirements.txt
```

Copy the repo-root `.env` (it already has your API keys):

```
cp ../.env .env
```

Required settings:

Variable	Meaning
`GEMINI_API_KEY`	API key for the LLM judge (Google AI Studio)
`STT_MODEL_NAME`	Whisper model size (default `base`)
`EVAL_MODEL_NAME`	the judge model (default `gemini-3.5-flash`)

Run

CLI — compare an audio file against a gold transcript:

```
python -m approach_1.main evaluate path/to/audio.ogg --gold
path/to/gold.txt
```

CLI — compare two text files directly (no audio):

```
python -m approach_1.main evaluate-text --gold gold.txt --candidate
candidate.txt
```

Web UI:

```
uvicorn approach_1.api:app --port 8000
```

Open http://localhost:8000, upload an audio file + a `.pdf`/`.txt` transcript, and click Run.

Output

The report contains:

- **overall_score** (0–100) and **status** (`Match` or `Mismatch`)
- **findings**, grouped by category — each with the gold text, the STT text, a severity (`low`/`medium`/`high`), and an explanation
- **meta** — how many LLM calls were used, latency, timestamp

Code layout

```
approach_1/
├── main.py         CLI
├── api.py          FastAPI web server
├── compare.py      interactive runner (pick audio + transcript)
├── config.py       reads .env settings
├── static/         web UI
├── datasets/       audio, gold transcripts, cached STT output
└── src/
```

```
    ├── evaluate.py   pipeline orchestrator
    ├── align.py      splits & matches texts
    ├── classify.py   LLM (or heuristic) classification
    ├── signals.py    similarity signals
    ├── score.py      final score + status
    ├── evaluator.py  Whisper STT + Gemini judge
    └── models.py     report data models
```

Tests

```
pytest
```